

第二十一讲：全连接神经网络

1 课程回顾：Softmax 回归与交叉熵损失

1.1 从二分类到多分类

在二分类问题中，Logistic 回归将一个线性函数

$$z = \mathbf{w}^\top \mathbf{x} + b$$

通过 Sigmoid 函数映射为概率

$$p(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

若类别数为 $K > 2$ ，我们希望模型同时输出 K 个类别的概率。Softmax 回归正是 Logistic 回归在多分类情形下的自然推广。

定义 1.1 (Softmax 函数). 设 $\mathbf{z} = (z_1, \dots, z_K)^\top \in \mathbb{R}^K$ ，Softmax 函数定义为

$$\text{softmax}(\mathbf{z})_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K.$$

显然 $\text{softmax}(\mathbf{z})_k > 0$ ，且 $\sum_{k=1}^K \text{softmax}(\mathbf{z})_k = 1$ 。

Softmax 回归模型通常写为

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}),$$

其中 $W \in \mathbb{R}^{K \times d}$ ， $\mathbf{b} \in \mathbb{R}^K$ 。 z_k 可理解为模型给第 k 类打出的未归一化分数，Softmax 将这些分数转化为概率。

注 1.1 (平移不变性与数值稳定). 对任意常数 c ，

$$\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c\mathbf{1}).$$

因此实际计算时常令 $c = -\max_k z_k$ ，即计算

$$\text{softmax}(\mathbf{z})_k = \frac{\exp(z_k - \max_j z_j)}{\sum_{r=1}^K \exp(z_r - \max_j z_j)},$$

以避免指数函数溢出。

定义 1.2 (交叉熵损失). 若真实标签采用 one-hot 向量 $\mathbf{y} \in \{0, 1\}^K$ 表示, 预测概率为 $\hat{\mathbf{y}}$, 则单个样本上的交叉熵损失为

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k.$$

若样本真实类别为 c , 则 $\ell = -\log \hat{y}_c$ 。

交叉熵损失有一个重要特点: 当模型给真实类别的概率越大时, 损失越小; 当模型把真实类别概率压得很低时, 损失会迅速变大。这与最大似然估计完全一致。若 $p_\theta(y = k | \mathbf{x}) = \hat{y}_k$, 则对独立同分布样本最大化似然等价于最小化平均交叉熵。

1.2 Softmax 与交叉熵的梯度

命题 1.1 (Softmax 的雅可比矩阵). 记 $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$ 。则

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j),$$

其中 δ_{ij} 为 Kronecker 符号。

证明. 由

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{r=1}^K \exp(z_r)}$$

直接求导。当 $i = j$ 时,

$$\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i).$$

当 $i \neq j$ 时,

$$\frac{\partial \hat{y}_i}{\partial z_j} = -\hat{y}_i \hat{y}_j.$$

合并即得结论。 □

命题 1.2 (Softmax 与交叉熵组合后的梯度). 设 $\mathbf{z} = W\mathbf{x} + \mathbf{b}$, $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$, 交叉熵损失为 $\ell = -\sum_k y_k \log \hat{y}_k$ 。则

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}.$$

因此

$$\frac{\partial \ell}{\partial W} = (\hat{\mathbf{y}} - \mathbf{y})\mathbf{x}^\top, \quad \frac{\partial \ell}{\partial \mathbf{b}} = \hat{\mathbf{y}} - \mathbf{y}.$$

注 1.2. 公式 $\partial \ell / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ 是深度学习中最常见的梯度公式之一。它说明分类器最后一层的误差信号就是“预测概率减真实标签”。

2 从线性模型到神经网络

2.1 线性模型的能力边界

线性模型的形式为

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

分类时，其决策边界为

$$\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\},$$

这是一个超平面。线性模型容易训练、解释性强、过拟合风险相对较小，但它无法直接表达复杂的非线性关系。

例 2.1 (XOR 问题). 考虑二元输入 $\mathbf{x} = (x_1, x_2) \in \{0, 1\}^2$ ，标签定义为

$$y = x_1 \oplus x_2.$$

即

$$(0, 0) \mapsto 0, \quad (1, 0) \mapsto 1, \quad (0, 1) \mapsto 1, \quad (1, 1) \mapsto 0.$$

在平面上，正类点与负类点交错分布，不存在一条直线可以把二者完全分开。因此单个线性分类器无法解决 XOR 问题。

神经网络的基本想法是：先通过若干可学习的非线性变换把原始输入映射到新的特征空间，再在新的特征空间中使用简单分类器或回归器。也就是说，网络同时学习

$$\text{特征表示} \quad \mathbf{h} = g(\mathbf{x}; \theta_g)$$

和

$$\text{预测函数} \quad \hat{y} = q(\mathbf{h}; \theta_q).$$

2.2 组合线性变换为什么不够

若网络的每一层都是线性变换，例如

$$\mathbf{h} = W_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{o} = W_2 \mathbf{h} + \mathbf{b}_2,$$

则

$$\mathbf{o} = W_2 W_1 \mathbf{x} + W_2 \mathbf{b}_1 + \mathbf{b}_2,$$

仍然只是关于 $\mathbf{x}$ 的线性函数。即使堆叠很多层，只要中间没有非线性激活函数，整个模型仍可等价为一个线性模型。因此，非线性激活函数不是装饰，而是神经网络表达能力的来源。

3 人工神经元

神经网络由大量人工神经元以及神经元之间的有向连接构成。与生物神经元的粗略类比是：树突接收输入，细胞体进行信息处理，轴突传递输出。人工神经元将这个过程抽象为“加权求和后通过激活函数”。

定义 3.1 (人工神经元). 给定输入 $\mathbf{x} = (x_1, \dots, x_d)^\top$, 权重 $\mathbf{w} = (w_1, \dots, w_d)^\top$ 和偏置 b , 神经元的净输入为

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b.$$

若激活函数为 $\phi: \mathbb{R} \rightarrow \mathbb{R}$, 则该神经元的输出为

$$a = \phi(z) = \phi(\mathbf{w}^\top \mathbf{x} + b).$$

其中, 权重 $\mathbf{w}$ 控制不同输入分量的重要性, 偏置 b 控制神经元被激活的阈值或平移位置。激活函数 ϕ 则决定神经元对净输入的响应方式。

注 3.1 (阈值视角). 若使用阶跃函数

$$\phi(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0, \end{cases}$$

则神经元只判断 $\mathbf{w}^\top \mathbf{x} + b$ 是否超过阈值。这种模型直观, 但不连续, 几乎处处梯度为 0, 不适合用梯度下降训练深层网络。

例 3.1 (一个二维神经元的几何意义). 设 $\mathbf{x} = (x_1, x_2)^\top$, $z = w_1 x_1 + w_2 x_2 + b$ 。直线

$$w_1 x_1 + w_2 x_2 + b = 0$$

把平面分成两部分。若使用 ReLU, 直线一侧输出为 0, 另一侧输出为线性函数。因此一个 ReLU 神经元可以看作对输入空间做了一次“折叠”或“截断”。

4 常见激活函数

一个实用的激活函数通常应满足以下性质：

1. 非线性, 否则多层网络会退化为线性模型。
2. 连续并尽量可导, 便于使用梯度优化。
3. 函数值和导数计算简单, 适合大规模训练。
4. 导数范围适中, 避免梯度在层间传播时过快消失或爆炸。
5. 输出分布较稳定, 避免每层输出过度偏移或饱和。

4.1 Sigmoid 型激活函数

定义 4.1 (Logistic 函数). Logistic 激活函数定义为

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

其导数为

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Logistic 函数输出位于 $(0, 1)$, 可解释为概率, 因此在二分类输出层中仍很常用。但在隐藏层中, 它有两个明显缺点: 第一, 输出不是零中心的; 第二, 当 z 很大或很小时, 导数接近 0, 容易造成梯度消失。

定义 4.2 (Tanh 函数). Tanh 激活函数定义为

$$\tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}.$$

其导数为

$$\frac{d}{dz} \tanh(z) = 1 - \tanh^2(z).$$

Tanh 的输出位于 $(-1, 1)$, 相比 Logistic 更接近零中心, 因此在某些网络中比 Logistic 更容易优化。不过它同样有饱和区, 深层网络中仍可能出现梯度消失。

注 4.1 (梯度消失的一个简单计算). 若某个深层网络的每一层都产生一个导数因子 0.25, 连续经过 20 层后, 误差信号的量级约为

$$0.25^{20} \approx 9.1 \times 10^{-13}.$$

这意味着靠近输入层的参数几乎收不到有效梯度。

4.2 ReLU 及其变体

定义 4.3 (ReLU 激活函数). ReLU (Rectified Linear Unit) 定义为

$$\text{ReLU}(z) = \max\{0, z\}.$$

其导数通常记为

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

在 $z = 0$ 处不可导, 实践中可取任意次梯度, 如 0 或 1。

ReLU 的优势包括:

1. 计算非常简单, 只需比较和取最大值。
2. 正半轴不饱和, 导数恒为 1, 比 Sigmoid 型函数更利于梯度传播。

3. 会产生精确的 0 输出，从而形成稀疏表示。
4. ReLU 网络整体是分段线性的，便于从几何上理解其表达能力。

注 4.2 (死亡神经元). 若某个 ReLU 神经元在所有训练样本上都满足 $z < 0$ ，则它的梯度恒为 0，参数无法继续更新。这种现象常被称为神经元“死亡”。初始化不当、学习率过大或数据分布偏移都可能增加其发生概率。

定义 4.4 (LeakyReLU 与 PReLU). LeakyReLU 在负半轴保留一个很小的斜率：

$$\phi(z) = \begin{cases} z, & z \geq 0, \\ \alpha z, & z < 0, \end{cases} \quad 0 < \alpha \ll 1.$$

PReLU (Parametric ReLU) 进一步把 α 作为可学习参数。

定义 4.5 (ELU、Softplus 与 Swish). ELU (Exponential Linear Unit) 定义为

$$\phi(z) = \begin{cases} z, & z \geq 0, \\ \alpha(\exp(z) - 1), & z < 0. \end{cases}$$

Softplus 是 ReLU 的平滑近似：

$$\phi(z) = \log(1 + \exp(z)).$$

Swish 函数可写为

$$\phi(z) = z \sigma(\beta z),$$

其中 β 可以是可学习参数，也可以是固定超参数。当 β 很大时，Swish 的形状接近 ReLU。

激活函数	优点	主要问题
Logistic	可解释为概率，平滑可导	非零中心，饱和区梯度很小
Tanh	零中心，平滑可导	饱和区仍会造成梯度消失
ReLU	计算快，正半轴不饱和，稀疏激活	负半轴梯度为 0，可能死亡
Leaky/PReLU	缓解死亡神经元	需要选择或学习负半轴斜率
Softplus/Swish	平滑，部分场景表现好	计算相对复杂，默认性不如 ReLU

5 网络结构

神经网络的核心除了神经元本身，还包括神经元之间的拓扑结构。常见结构可以粗略分为三类：

1. 前馈神经网络：信息沿一个方向传播，从输入层经过若干隐藏层到达输出层。

2. 记忆网络或反馈网络：神经元具有状态，网络在不同时刻具有不同的内部表示。
3. 图网络：定义在图结构数据上，用节点、边及其邻接关系组织信息传播。

本讲重点讨论前馈神经网络。全连接神经网络是最基本的前馈神经网络。

定义 5.1 (全连接前馈神经网络). 设网络共有 L 层，其中第 0 层为输入层，第 L 层为输出层。记第 l 层神经元个数为 m_l 。若相邻两层之间的神经元两两相连，层内无连接，且无反馈连接，则称该网络为全连接前馈神经网络。

全连接网络中的“全连接”指第 $l-1$ 层的每个输出都连接到第 l 层的每个神经元。因此，第 l 层权重矩阵包含 $m_l m_{l-1}$ 个权重，偏置向量包含 m_l 个偏置。

例 5.1 (参数量计算). 考虑输入维度 784、两个隐藏层宽度分别为 256, 128、输出类别数为 10 的全连接网络。参数量为

$$256 \times 784 + 256 + 128 \times 256 + 128 + 10 \times 128 + 10 = 234762.$$

这个例子说明，全连接层参数量随相邻两层宽度的乘积增长。

6 前馈神经网络的数学表示

设输入为 $\mathbf{x} \in \mathbb{R}^{m_0}$ 。记第 l 层的净输入为 $\mathbf{z}^{(l)} \in \mathbb{R}^{m_l}$ ，输出（活性值）为 $\mathbf{a}^{(l)} \in \mathbb{R}^{m_l}$ 。令

$$\mathbf{a}^{(0)} = \mathbf{x}.$$

对 $l = 1, \dots, L$ ，前馈计算为

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}),$$

其中

$$W^{(l)} \in \mathbb{R}^{m_l \times m_{l-1}}, \quad \mathbf{b}^{(l)} \in \mathbb{R}^{m_l}.$$

这里 $\phi^{(l)}$ 通常逐分量作用于向量 $\mathbf{z}^{(l)}$ 。

整个网络可以看作一个复合函数

$$f(\mathbf{x}; \theta) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(\mathbf{x}),$$

其中

$$\theta = \{W^{(l)}, \mathbf{b}^{(l)} : l = 1, \dots, L\}$$

表示网络中所有可学习参数。

6.1 小批量矩阵形式

实际训练时通常不是一次输入一个样本，而是输入一个小批量。设批量大小为 B ，把样本按列组成矩阵

$$A^{(0)} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_B \end{bmatrix} \in \mathbb{R}^{m_0 \times B}.$$

则第 l 层前馈计算可写为

$$Z^{(l)} = W^{(l)} A^{(l-1)} + \mathbf{b}^{(l)} \mathbf{1}^\top, \quad A^{(l)} = \phi^{(l)}(Z^{(l)}).$$

这种写法能充分利用矩阵乘法，提高训练效率。

例 6.1 (单隐藏层分类网络). 若网络只有一层隐藏层，隐藏层宽度为 m ，输出层维度为 K ，则

$$\mathbf{h} = \phi(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{o} = W_2 \mathbf{h} + \mathbf{b}_2,$$

分类任务中通常再令

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{o}).$$

此时隐藏层 $\mathbf{h}$ 可以看作从原始输入中学习到的特征表示，输出层则是作用在该表示上的 Softmax 回归分类器。

7 应用到机器学习

神经网络可以看作一个可学习的“万能”函数。一方面，它可以用来完成复杂的特征转换；另一方面，它也可以直接逼近输入到输出之间的条件分布。

7.1 深度线性回归

考虑回归数据集

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R}.$$

若采用一层隐藏层的全连接网络，

$$\mathbf{h}_i = \phi(W_1 \mathbf{x}_i + \mathbf{b}_1), \quad \hat{y}_i = \mathbf{w}_2^\top \mathbf{h}_i + b_2,$$

则常用平方损失作为学习准则：

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2.$$

注 7.1. 这里的“深度线性回归”并不是说整个模型是线性的，而是说最后一层是线性回归器。隐藏层负责学习非线性特征，最后一层负责把特征线性组合成预测值。

7.2 深度 Softmax 回归

对多分类问题，神经网络的最后一层可以看作 Softmax 回归分类器。若输出层有 K 个神经元，则

$$\mathbf{o}_i = W_L \mathbf{a}_i^{(L-1)} + \mathbf{b}_L, \quad \hat{\mathbf{y}}_i = \text{softmax}(\mathbf{o}_i).$$

采用交叉熵损失时，经验风险为

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \left[- \sum_{k=1}^K y_{ik} \log \hat{y}_{ik} \right].$$

从这个角度看，深度分类网络就是

多层特征提取器 + Softmax 分类器.

7.3 用于聚类

神经网络也可用于聚类。一个常见思路是先用深度网络学习表示

$$\mathbf{h}_i = f(\mathbf{x}_i; \theta),$$

再在表示空间中进行 K -means 聚类：

$$\min_{\theta, \{\mu_k\}} \sum_{i=1}^n \min_{1 \leq k \leq K} \|\mathbf{h}_i - \mu_k\|_2^2.$$

此时网络的作用是把原始数据映射到更适合聚类的特征空间。

注 7.2 (哪里需要哪里搬). 在实际应用中，神经网络经常作为一个可学习的特征转换模块嵌入传统机器学习流程：分类时接 Softmax，回归时接线性输出，聚类时接 K -means，排序时接排序损失。深度学习的灵活性很大程度上来自这种模块化观点。

8 表达能力：通用逼近与深度的意义

定理 8.1 (通用逼近定理，非严格表述). 在适当条件下，含有一层隐藏层、使用非线性激活函数的前馈神经网络，可以在紧集上以任意精度逼近连续函数。

通用逼近定理说明一层隐藏层网络在表达能力上是可行的，但它并不说明这种表示一定高效。傅里叶基、多项式基、高斯核等方法也具有类似的逼近能力。深度学习真正关心的是：在有限样本、有限计算资源和可优化性的约束下，什么样的模型结构更容易学到有效表示。

8.1 为什么需要多层

直观地说，多层网络通过逐层组合低级特征来形成高级特征。例如图像任务中，浅层可能提取边缘和纹理，中间层组合成局部形状，高层进一步组合成物体部件或类别相

关特征。虽然全连接网络不像卷积网络那样显式利用空间结构，但“逐层组合”的思想仍然成立。

注 8.1 (表达效率). 某些函数用浅层网络逼近时可能需要指数级数量的神经元，而用多层网络可以用远少得多的神经元表示。这类结论说明，深度不仅增加参数数量，更改变了函数族的组合结构。

8.2 ReLU 网络的分段线性观点

若所有隐藏层都使用 ReLU，则网络整体是分段线性函数。每个 ReLU 神经元根据 $z > 0$ 或 $z < 0$ 把输入空间分为激活区和非激活区。多个 ReLU 神经元叠加后，输入空间被划分为许多区域，在每个区域内网络都是一个线性函数，但不同区域对应不同的线性函数。

例 8.1 (一维 ReLU 组合). 函数

$$f(x) = a_0 + a_1 x + \sum_{j=1}^m c_j \text{ReLU}(x - t_j)$$

是一个一维分段线性函数。每个 $\text{ReLU}(x - t_j)$ 都在 $x = t_j$ 处引入一个折点。因此，多个 ReLU 的线性组合可以逼近许多一维连续函数。

9 参数学习与经验风险

给定训练集

$$D = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n,$$

将每个样本输入前馈神经网络，得到输出

$$\hat{\mathbf{y}}_i = f(\mathbf{x}_i; \theta).$$

若损失函数为 $\ell(\mathbf{y}, \hat{\mathbf{y}})$ ，正则项为 $\Omega(\theta)$ ，则结构化风险可写为

$$R(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{y}_i, f(\mathbf{x}_i; \theta)) + \lambda \Omega(\theta).$$

参数学习通常通过梯度下降及其变体完成：

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} R(\theta_t),$$

其中 η_t 为学习率。

定义 9.1 (随机梯度下降). 每次只用一个小批量 B_t 估计整体梯度：

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \ell(\mathbf{y}_i, f(\mathbf{x}_i; \theta_t)).$$

更新为

$$\theta_{t+1} = \theta_t - \eta_t g_t.$$

随机梯度下降的单步方向带有噪声，但计算代价低，且噪声在非凸优化中有时有助于逃离较差区域。实际深度学习训练常使用带动量的 SGD、Adam、AdamW 等变体。

10 反向传播：链式法则的系统化应用

深度网络本质上是多层复合函数。设

$$f = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}.$$

根据链式法则，梯度会在层间连续相乘：

$$\frac{\partial f}{\partial \mathbf{x}} = \frac{\partial f^{(L)}}{\partial \mathbf{a}^{(L-1)}} \frac{\partial f^{(L-1)}}{\partial \mathbf{a}^{(L-2)}} \dots \frac{\partial f^{(1)}}{\partial \mathbf{x}}.$$

反向传播算法的作用是高效组织这些链式法则计算，避免对每个参数重复做大量公共计算。

10.1 单样本 MLP 的反向传播公式

考虑前馈计算

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \phi^{(l)}(\mathbf{z}^{(l)}).$$

设损失为 ℓ 。定义第 l 层误差信号

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \ell}{\partial \mathbf{z}^{(l)}}.$$

输出层的 $\boldsymbol{\delta}^{(L)}$ 由具体损失函数决定。例如 Softmax 加交叉熵时，

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}.$$

对隐藏层 $l = L - 1, \dots, 1$ ，有

$$\boldsymbol{\delta}^{(l)} = ((W^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}) \odot \phi'^{(l)}(\mathbf{z}^{(l)}),$$

其中 $\odot$ 表示逐元素乘法。参数梯度为

$$\frac{\partial \ell}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad \frac{\partial \ell}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}.$$

注 10.1 (反向传播不是一种新的微积分)。反向传播本质上是链式法则在计算图上的高效实现。它的关键不是公式本身有多神秘，而是它把许多中间量缓存下来，使得梯度计算的复杂度与前馈计算同阶。

例 10.1 (两层网络的梯度)。设

$$\mathbf{h} = \phi(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{o} = W_2 \mathbf{h} + \mathbf{b}_2, \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{o}).$$

交叉熵损失下,

$$\boldsymbol{\delta}^{(2)} = \hat{\mathbf{y}} - \mathbf{y}.$$

于是

$$\frac{\partial \ell}{\partial W_2} = \boldsymbol{\delta}^{(2)} \mathbf{h}^\top, \quad \frac{\partial \ell}{\partial \mathbf{b}_2} = \boldsymbol{\delta}^{(2)}.$$

隐藏层误差为

$$\boldsymbol{\delta}^{(1)} = W_2^\top \boldsymbol{\delta}^{(2)} \odot \phi'(W_1 \mathbf{x} + \mathbf{b}_1),$$

因此

$$\frac{\partial \ell}{\partial W_1} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top, \quad \frac{\partial \ell}{\partial \mathbf{b}_1} = \boldsymbol{\delta}^{(1)}.$$

11 初始化、梯度消失与梯度爆炸

深度网络训练的困难之一来自多层雅可比矩阵的连乘。若每层传播都让信号方差变小, 深层之后信号趋近于 0; 若每层都让方差变大, 信号会爆炸。梯度反向传播时也会遇到类似问题。

11.1 Xavier 初始化的直观推导

考虑线性层

$$z_i = \sum_{j=1}^{n_{\text{in}}} w_{ij} x_j.$$

若假设 w_{ij} 与 x_j 独立, 均值为 0, 且 x_j 方差相同, 则

$$\text{Var}(z_i) \approx n_{\text{in}} \text{Var}(w_{ij}) \text{Var}(x_j).$$

为了让前向传播中方差大致保持不变, 需要

$$n_{\text{in}} \text{Var}(w_{ij}) \approx 1.$$

同时考虑反向传播时的方差稳定, 可得到与

$$\text{Var}(w_{ij}) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

相近的初始化原则, 这就是 Xavier 初始化的核心思想。

11.2 He 初始化

若激活函数为 ReLU, 大约一半输入会被截断为 0。为补偿这种方差损失, 常使用

$$\text{Var}(w_{ij}) \approx \frac{2}{n_{\text{in}}}.$$

这被称为 He 初始化或 Kaiming 初始化, 特别适合 ReLU/PReLU 网络。

注 11.1 (初始化的目的). 初始化不是为了让网络一开始预测得好, 而是为了让信号和梯度能在很多层之间稳定传播。好的初始化让优化从一个比较健康的区域开始。

12 为什么不总是过拟合

现代深度网络的参数量经常大于样本量。从经典统计学习的直觉看，这似乎必然导致过拟合。但实际中，深度网络在许多任务上能够同时做到训练误差低和测试误差低。这一现象仍有许多开放解释。

常见理解包括：

1. 数据结构：真实数据通常集中在低维流形或具有强结构的区域，而不是均匀填满高维空间。
2. 表示学习：网络学习到的中间表示可能抽取了任务相关因素，过滤了无关噪声。
3. 优化偏置：即使参数空间很大，梯度下降也不会均匀搜索整个空间，而倾向于某些较简单或较平滑的解。
4. 正则化机制：权重衰减、数据增强、早停、Dropout、批归一化等都会影响最终解。
5. 过参数化：参数多有时反而让优化更容易，因为存在许多能很好拟合训练集的路径。

注 12.1. “参数多”并不等同于“有效复杂度一定大”。深度学习理论中的一个核心问题，就是如何刻画优化算法、网络结构和数据分布共同决定的有效模型复杂度。